

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

«Контейнеризация приложения средствами Docker»

Выполнил:

Якшин Артемий

Группа БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Контейнеризировать микросервисное приложение «Restaurant Booking» из ЛР2 средствами Docker: реализовать `Dockerfile` для каждого сервиса, написать общий `docker-compose.yml`, настроить сетевое взаимодействие между сервисами. Результат — единая команда поднимает весь стек, сервисы находят друг друга по сети, наружу публикуется только шлюз.

Ход работы

1. Что контейнеризируется

Пять сервисов из ЛР2: `gateway` (порт 3000), `auth-service` (8081), `catalog-service` (8082), `reservation-service` (8083), `review-service` (8084). У каждого — отдельная SQLite-БД, которая в Docker хранится в именованном volume (`database-per-service`).

2. Dockerfile для каждого сервиса

Для всех сервисов используется единый шаблон образа (базовый `node:20` — содержит `toolchain` для нативного модуля `sqlite3`): установка зависимостей, копирование исходников, компиляция TypeScript (`npm run build` → `tsc`), запуск собранного приложения.

```
FROM node:20
WORKDIR /app

COPY package.json ./
RUN npm install                # слой кэшируется, пока не менялся package.json

COPY tsconfig.json ./
COPY src ./src
RUN npm run build              # tsc: src -> dist

ENV NODE_ENV=production
EXPOSE 8082                    # порт сервиса (у каждого свой:
3000/8081/8082/8083/8084)
CMD ["node", "dist/index.js"]
```

Рядом с каждым `Dockerfile` лежит `.dockerignore` (исключает `node_modules`, `dist`, `*.sqlite`, `.env`, `.git`), чтобы в контекст сборки не попадал мусор и локальные артефакты.

3. Общий docker-compose.yml

Один файл описывает все 5 сервисов: для каждого — `build` (путь к каталогу с `Dockerfile`), переменные окружения (порт, путь к БД, секреты, адреса соседей по DNS-именам), у бэкенд-сервисов — именованный `volume` для БД, общая `network`,

`healthcheck` и `depends_on` с условием `service_healthy`. Наружу публикуется только порт шлюза (`${GATEWAY_PORT:-3000}:3000`); остальные сервисы доступны лишь внутри сети.

```

services:
  auth-service:
    build: ./auth-service
    environment:
      PORT: 8081
      DB_PATH: /data/auth.sqlite
      JWT_SECRET: ${JWT_SECRET:-supersecretjwtkey}
      INTERNAL_API_KEY: ${INTERNAL_API_KEY:-internal-secret}
    volumes: [auth-data:/data]
    networks: [rbnet]
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://localhost:8081/health').then(r=>process.exit(r.ok?
0:1)).catch(()=>process.exit(1))"]
      interval: 10s
      timeout: 3s
      retries: 6

  catalog-service:
    build: ./catalog-service
    environment:
      PORT: 8082
      DB_PATH: /data/catalog.sqlite
      REVIEW_URL: http://review-service:8084
      RESERVATION_URL: http://reservation-service:8083
      INTERNAL_API_KEY: ${INTERNAL_API_KEY:-internal-secret}
    volumes: [catalog-data:/data]
    networks: [rbnet]
    healthcheck: { test: [...], interval: 10s, timeout: 3s, retries: 6 }

  reservation-service:
    build: ./reservation-service
    environment:
      PORT: 8083
      DB_PATH: /data/reservations.sqlite
      JWT_SECRET: ${JWT_SECRET:-supersecretjwtkey}
      CATALOG_URL: http://catalog-service:8082
      INTERNAL_API_KEY: ${INTERNAL_API_KEY:-internal-secret}
    volumes: [reservation-data:/data]
    networks: [rbnet]
    depends_on: { catalog-service: { condition: service_healthy } }
    healthcheck: { ... }

  review-service:
    build: ./review-service
    environment:
      PORT: 8084
      DB_PATH: /data/reviews.sqlite
      JWT_SECRET: ${JWT_SECRET:-supersecretjwtkey}
      CATALOG_URL: http://catalog-service:8082
      AUTH_URL: http://auth-service:8081
      INTERNAL_API_KEY: ${INTERNAL_API_KEY:-internal-secret}
    volumes: [review-data:/data]
    networks: [rbnet]
    depends_on:
      catalog-service: { condition: service_healthy }
      auth-service: { condition: service_healthy }
    healthcheck: { ... }

  gateway:
    build: ./gateway
    environment:
      PORT: 3000
      AUTH_URL: http://auth-service:8081

```

```
CATALOG_URL: http://catalog-service:8082
RESERVATION_URL: http://reservation-service:8083
REVIEW_URL: http://review-service:8084
ports: ["${GATEWAY_PORT:-3000}:3000"]
networks: [rbnet]
depends_on:
  auth-service:      { condition: service_healthy }
  catalog-service:   { condition: service_healthy }
  reservation-service: { condition: service_healthy }
  review-service:    { condition: service_healthy }

volumes: { auth-data: , catalog-data: , reservation-data: , review-data: }
networks: { rbnet: { driver: bridge } }
```

4. Сетевое взаимодействие

- Все контейнеры подключены к одной пользовательской bridge-сети `rbnet`; внутри неё работает встроенный DNS Docker — сервис обращается к соседу по имени из `compose`: `http://catalog-service:8082`, `http://auth-service:8081` и т. д. (адреса заданы переменными окружения).
- Наружу (на хост) проброшен **только** порт шлюза: `${GATEWAY_PORT:-3000}:3000`. Порты `8081-8084` объявлены через `EXPOSE`, но не публикуются — бэкенд-сервисы и их внутренние эндпоинты `/internal/*` снаружи недоступны.
- Порядок старта: `depends_on` + `condition: service_healthy` (на базе `healthcheck`, который дёргает `/health` сервиса) — `gateway` и зависящие сервисы стартуют только после готовности тех, на кого они опираются.
- Данные каждого сервиса — в своём именованном `volume` (`auth-data`, `catalog-data`, `reservation-data`, `review-data`), смонтированном в `/data`; переживают пересоздание контейнеров.

5. Запуск и проверка

```

$ cd "БР1.1/Якшин Артемий/labs/lab2"
$ docker compose build
... lab2-auth-service:latest, lab2-catalog-service:latest, lab2-reservation-
service:latest,
    lab2-review-service:latest, lab2-gateway:latest  - DONE

$ docker compose up -d
lab2-catalog-service-1  Healthy
lab2-auth-service-1     Healthy
lab2-review-service-1   Healthy
lab2-reservation-service-1 Healthy
lab2-gateway-1          Started

$ docker compose ps
NAME                                SERVICE                                STATUS                                PORTS
lab2-auth-service-1                auth-service                           Up (healthy)                          8081/tcp
lab2-catalog-service-1             catalog-service                         Up (healthy)                          8082/tcp
lab2-reservation-service-1         reservation-service                    Up (healthy)                          8083/tcp
lab2-review-service-1              review-service                         Up (healthy)                          8084/tcp
lab2-gateway-1                     gateway                                Up
0.0.0.0:3000->3000/tcp

$ docker volume ls | grep lab2
local    lab2_auth-data
local    lab2_catalog-data
local    lab2_reservation-data
local    lab2_review-data
$ docker network ls | grep lab2
...     lab2_rbnnet    bridge    local

# проверка снаружи — только через шлюз:
$ curl -s http://localhost:3000/health
{"status":"ok","service":"api-gateway"}

$ TOKEN=$(curl -s -X POST http://localhost:3000/api/v1/auth/login \
  -d '{"email":"ivan@example.com","password":"securepass123"}' | jq -r .token)
$ curl -s --get --data-urlencode "city=Москва" http://localhost:3000/api/v1/restaurants
| jq '.total'
3
$ curl -s "http://localhost:3000/api/v1/restaurants/1/tables?date=2026-07-
01&time=19:00" | jq 'length'
5
$ curl -s -X POST http://localhost:3000/api/v1/reservations -H "Authorization: Bearer
$TOKEN" \
  -d '{"table_id":1,"reservation_date":"2026-07-
01","reservation_time":"19:00","guests_count":2}' | jq '.status'
"confirmed"
$ curl -s -X POST http://localhost:3000/api/v1/reviews -H "Authorization: Bearer
$TOKEN" \
  -d '{"restaurant_id":2,"rating":5,"comment":"Отлично!"}' | jq '.review_id'
4
# (внутренние эндпоинты соседей при этом вызывались внутри сети rbnnet:
# reservation→catalog /internal/tables, review→catalog /internal/restaurants,
review→auth /internal/users/batch,
# catalog→review /internal/ratings/batch, catalog→reservation
/internal/reservations/availability-batch)

$ docker compose down          # остановить; down -v — вместе с данными (volumes)

```

Весь стек поднимается одной командой `docker compose up --build`; все 5 контейнеров переходят в состояние *healthy*; сквозной сценарий (вход → каталог с рейтингами → столики → бронирование → отзыв) проходит через шлюз и

задействует межсервисные вызовы внутри сети `rbnet`; снаружи доступен только порт 3000.

Замечание. Образы собраны на базе полного `node:20` (~1.8 ГБ каждый): он содержит компиляторы, нужные для сборки нативного модуля `sqlite3`. При желании размер можно существенно сократить multi-stage-сборкой (этап `node:20` для `npm install + tsc`, затем копирование `dist` и `node_modules` в тонкий `node:20-slim`); для целей данной работы это не требовалось.

Вывод

Микросервисное приложение «Restaurant Booking» контейнеризировано средствами Docker: для каждого из пяти сервисов написан `Dockerfile` (единый шаблон на базе `node:20` с компиляцией TypeScript), составлен общий `docker-compose.yml`, описывающий все сервисы, их переменные окружения, именованные `volume`'ы для баз данных, `healthcheck`'и и порядок старта через `depends_on/service_healthy`. Настроено сетевое взаимодействие: единая bridge-сеть `rbnet`, обращение сервисов друг к другу по DNS-именам, наружу проброшен только порт шлюза. Весь стек запускается одной командой и проходит сквозную проверку; данные сервисов изолированы и сохраняются в `volume`'ах. Работа служит основой для последующего развёртывания (JР4) и CI/CD (Д36).